

Сравнение наибольших общих префиксов [LCP-сравнение]

Алгоритмы сортировки общего назначения, которые не специализированы для работы со строками, демонстрируют низкую эффективность при сортировке текстовых данных. Это происходит из-за несбалансированности между затратами на сравнение строк и информативностью получаемого результата. При посимвольном сравнении строк требуется значительное время обработки, в то время как итоговый результат сравнения предоставляет лишь небольшое количество полезной информации — всего один бит (в случае двоичного сравнения) или один трит (при трех равновероятных исходах).

STRING QUICKSORT, представленный на лекции 25, решает обозначенную проблему путем немедленного использования результата каждого посимвольного сравнения.

Другой подход состоит в том, чтобы заменить обычное сравнение строк сравнением их **наибольших общих префиксов**. Определим функцию $\text{LcpCompare}(A, B, k)$:

- 1** возвращаемый результат — пара (x, l) , где $x \in \{<, =, >\}$ определяет лексикографический порядок между A и B , а $l = \text{LCP}(A, B)$ соответствует длине наибольшего общего префикса строк A и B .
- 2** третий параметр операции k соответствует длине известного общего префикса строк A и B , то есть представляет нижнюю границу $\text{LCP}(A, B)$. Таким образом, при сравнении A и B **можно пропустить** первые k символов.

В этом случае, дополнительное время, затрачиваемое на посимвольное сравнение строк, балансируется дополнительной информацией о длине их наибольшего общего префикса. Следует отметить, что **различающий префикс** $\text{DP}(A, B)$ и наибольший общий префикс $\text{LCP}(A, B)$ фактически ничем не отличаются.

Как показывают следующие соотношения, результаты известных сравнений строк могут быть полезны для получения нижней границы или точного значения наибольшего общего префикса:

- $\text{LCP}(A, C) \geq \min\{\text{LCP}(A, B), \text{LCP}(B, C)\}$.
- Если $A \leq B \leq C$, то $\text{LCP}(A, C) = \min\{\text{LCP}(A, B), \text{LCP}(B, C)\}$.
- Если $\text{LCP}(A, B) \neq \text{LCP}(B, C)$, то $\text{LCP}(A, C) = \min\{\text{LCP}(A, B), \text{LCP}(B, C)\}$.

Другими словами, длины наибольших общих префиксов трех строк A , B и C не могут быть тремя различными значениями. Как минимум два из них должны совпадать, тогда как третье будет таким же или большим.

Используя наибольший общий префикс, мы можем определить лексикографический порядок строк без непосредственного посимвольного сравнения. Предположим, что заданы четыре строки A , B_1 , B_2 и C , про которые известно, что $A \leq B_1 \leq C$ и $A \leq B_2 \leq C$. Тогда справедливы следующие симметричные соотношения:

- Если $\text{LCP}(A, B_1) > \text{LCP}(A, B_2)$, то $B_1 < B_2$.
- Если $\text{LCP}(B_1, C) > \text{LCP}(B_2, C)$, то $B_1 > B_2$.

Чем больше длина наибольшего общего префикса двух строк, тем они «**ближе друг к другу**» с точки зрения **лексикографического** порядка. Например, $\text{LCP}(\text{KOT}, \text{POT}) = 0$, а $\text{LCP}(\text{KOT}, \text{KPOT}) = 1$.

Сортировка слиянием для строк

Алгоритм STRING MERGESORT использует LCP-сравнения для сортировки строк в лексикографическом порядке. Он имеет такую же структуру: выполняется сортировка первой и второй половины набора строк по отдельности, а затем — их слияние:

```
ВХОД:   Множество строк  $R = \{S_1, S_2, \dots, S_n\}$ 
ВЫХОД:   $R$  в неубывающем лексикографическом порядке, дополненное
        значениями длин наибольших общих префиксов  $LCP_R$ 

if  $|R| = 1$  then return  $(S_1, 0)$ 

 $m \leftarrow n / 2$ 
 $P = \text{StringMergeSort}(\{S_1, S_2, \dots, S_m\})$ 
 $Q = \text{StringMergeSort}(\{S_{m+1}, S_{m+2}, \dots, S_n\})$ 

return  $\text{StringMerge}(P, Q)$ 
```

В результате работы алгоритма имеем упорядоченную в лексикографическом порядке последовательность пар $((T_1, \ell_1), (T_2, \ell_2), \dots, (T_n, \ell_n))$, где $\ell_i = LCP(T_i, T_{i-1})$ для $i > 1$, а $\ell_1 = 0$. Длины наибольших общих префиксов эффективно используются на этапе слияния строк **StringMerge**:

```
ВХОД:    $P = \{(S_1, k_1), (S_2, k_2), \dots, (S_m, k_m)\}$  и  $Q = \{(W_1, p_1), (W_2, p_2), \dots, (W_f, p_f)\}$ 
ВЫХОД:   $R$  — результат упорядоченного объединения  $P$  и  $Q$ 

 $R \leftarrow \emptyset; i = 1; j = 1$ 

while  $i \leq m$  and  $j \leq f$  do
    if  $k_i > p_j$  then
        добавить  $(S_i, k_i)$  к  $R$ 
         $i \leftarrow i + 1$ 
    else if  $k_i < p_j$  then
        добавить  $(W_j, p_j)$  к  $R$ 
         $j \leftarrow j + 1$ 
    else
         $(x, h) \leftarrow \text{LcpCompare}(S_i, W_j, k_i)$ 
        if  $x = <$  then
            добавить  $(S_i, k_i)$  к  $R$ 
             $i \leftarrow i + 1; p_j \leftarrow h$ 
        else
            добавить  $(W_j, p_j)$  к  $R$ 
             $j \leftarrow j + 1; k_i \leftarrow h$ 

while  $i \leq m$  do добавить  $(S_i, k_i)$  к  $R; i \leftarrow i + 1$ 
while  $j \leq f$  do добавить  $(W_j, p_j)$  к  $R; j \leftarrow j + 1$ 

return  $R$ 
```

В приведенном выше алгоритме для упрощения нотации индексов строкам из множества Q сопоставлены имена $W_1, \dots, W_f$.

Далее рассмотрим примеры выполнения нескольких шагов слияния строк. В начале работы алгоритма STRING MERGESORT происходит последовательное разбиение исходного множества строк, пока не останутся одноэлементные множества. Начальное значение k , которое используется при вычислении длины наибольших общих префиксов в $LcpCompare(A, B, k)$, равно 0.

P		Q							
S_1	k_1	S_2	k_2	S_3	k_3	S_4	k_4	S_5	k_5
ПАМКА	0	КОТ	0	КРОТ	0	КРАН	0	КРОП	0

Шаг 1. Выполним слияние $P = (S_1, k_1)$ с $Q = (S_2, k_2)$. В соответствии с алгоритмом **StringMerge** нам нужно найти длину их наибольшего общего префикса — $LcpCompare(S_1, S_2, k_1) = (>, 0)$, поскольку $S_1 > S_2$, а общего префикса строки не имеют. Тогда, мы добавляем (S_2, k_2) к результату слияния R и записываем $k_1 = 0$, а также добавляем к R остаток из P .

R				P		Q	
S_2	k_2	S_1	k_1	S_3	k_3	S_4	k_4
КОТ	0	ПАМКА	0	КРОТ	0	КРАН	0

Шаг 2. Выполним слияние $P = (S_4, k_4)$ с $Q = (S_5, k_5)$. В соответствии с алгоритмом **StringMerge** нам нужно найти длину их наибольшего общего префикса — $LcpCompare(S_4, S_5, k_4) = (<, 2)$, поскольку $S_4 < S_5$, а их наибольший общий префикс — КР. Тогда, мы добавляем (S_4, k_4) к результату слияния R , записываем $k_5 = 2$, а также добавляем к R остаток из Q .

R							
S_2	k_2	S_1	k_1	S_3	k_3	S_4	k_4
КОТ	0	ПАМКА	0	КРОТ	0	КРАН	0

S_4	k_4	S_5	k_5
КРАН	0	КРОП	2

Длина наибольшего общего префикса этой и «предыдущей» строки

Шаг 3. Выполним слияние $P = (S_3, k_3)$ с $Q = ((S_4, k_4), (S_5, k_5))$. В соответствии с алгоритмом **StringMerge** нам сначала нужно найти длину наибольшего общего префикса S_3 и S_4 — $LcpCompare(S_3, S_4, k_3) = (>, 2)$, поскольку $S_3 > S_4$, а их наибольший общий префикс — КР. Тогда, мы добавляем (S_4, k_4) к результату слияния, то есть $R = (S_4, k_4)$, и записываем $k_3 = 2$.

R				P		Q	
S_2	k_2	S_1	k_1	S_4	k_4	S_3	k_3
КОТ	0	ПАМКА	0	КРАН	0	КРОТ	2

Затем обрабатываем (S_3, k_3) и (S_5, k_5) . Поскольку $k_3 = k_5$, пересчитаем в соответствии с **StringMerge** длину их наибольшего общего префикса — $LcpCompare(S_3, S_5, k_3) = (>, 3)$, так как $S_3 > S_5$, а их наибольший общий префикс — КРО. Тогда, мы добавляем (S_5, k_5) к результату слияния, записываем $k_3 = 3$, а также добавляем к R остаток из P .

R							
S_2	k_2	S_1	k_1	S_4	k_4	S_5	k_5
КОТ	0	ПАМКА	0	КРАН	0	КРОП	2

S_3	k_3
КРОТ	3

Корректность и сложность сортировки слиянием для строк

Представленный алгоритм STRING MERGESORT корректно выполняет слияние сортируемых строк. Этот факт следует из того, что справедлив следующий инвариант цикла **while** алгоритма **StringMerge**:



Пусть X — это последняя строка, добавляемая в результат слияния R строк из множеств P и Q . Тогда $k_i = \text{LCP}(X, S_i)$, а $p_j = \text{LCP}(X, W_j)$.

Покажем, что приведенный инвариант остается верным, а также, что выбирается меньшая в лексикографическом порядке строка на каждой итерации цикла **while**:

- Если $k_i > p_j$, то $\text{LCP}(X, S_i) > \text{LCP}(X, W_j)$ и, по основным свойствам наибольшего общего префикса, $\text{LCP}(S_i, W_j) = \text{LCP}(X, W_j)$, потому что $\text{LCP}(X, W_j) = \min\{\text{LCP}(X, S_i), \text{LCP}(S_i, W_j)\}$. Поэтому, алгоритм выберет меньшую в лексикографическом порядке строку. Случай $k_i > p_j$ симметричен.
- Если $k_i = p_j$, то $\text{LCP}(S_i, W_j) > k_i$ и обращение к `LcpCompare` безопасно. Изменение $k_i = h$ или $p_j = h$ сохраняют инвариант.

STRING MERGESORT является оптимальным алгоритмом сортировки строк на основе посимвольных сравнений и имеет временную сложность, которая зависит от суммарной длины наибольших общих префиксов всех строк в R , $O(\sum \text{LCP}(R) + n \cdot \log n)$:

- Если обращение к `LcpCompare` занимает постоянное время, то временная сложность определяется стандартной границей сортировок на основе сравнений — $O(n \cdot \log n)$.
- Если обращение к `LcpCompare` требует $t + 1$ посимвольных сравнений, то одно из значений длин наибольших префиксов, ассоциированных со строкой, увеличивается на t . Поскольку суммарная длина общих префиксов в точности равна $\sum \text{LCP}(R)$, то дополнительные временные затраты `LcpCompare` ограничены сверху $O(\sum \text{LCP}(R))$.